GUIA DE ESTUDO

Como funciona o código do Marvel Defence

Da entrada no Android ao resultado da quinta onda. Exemplos extraidos dos arquivos atuais, com linhas para acompanhar no Android Studio.

> Objetivo Dar uma visão funcional do projeto. Você vai identificar onde cada regra vive, seguir um toque até o motor e entender como o estado volta para a tela.

Parte	O que estudar
1. Estrutura	Pastas, arquivos e responsabilidades.
2. Interface	XML, recursos R, Views e navegação.
3. Heróis	Catálogo, seleção, unidade e desenho.
4. Cenário	Imagem, geometria, coordenadas e toque.
5. Partida	Posicionamento, tempo, ondas, ataques e resultado.
6. Apoio	Comic Vine, animações, ciclo de vida e testes.

Como usar: leia uma parte por vez, abra as linhas indicadas e faça o exercício da seção. Não precisa decorar o código; procure entender quem recebe a ação, quem decide o resultado e quem desenha.

1. Mapa geral do projeto

O app tem uma Activity principal. Ela coordena as telas e os comandos. O pacote game executa as regras em Java puro. O pacote ui desenha e recebe interação. O pacote data define heróis e fichas. A pasta res contém layouts, imagens, textos e estilos.

Arquivo ou grupo	Responsabilidade
AndroidManifest.xml	Diz ao Android que MainActivity é a tela de entrada; fixa orientação horizontal.
MainActivity.java	Troca telas, conecta botões, equipe, motor, HUD e ciclo de atualização.
data/HeroCatalog.java	Seis heróis jogáveis e valores provisórios de combate.
data/ComicVineRepository.java	Fichas informativas, cache e atualização manual da API.
game/GameConfig.java	Constantes de ondas, economia, colocação e tempo.
game/MapData.java	Rua, pontos do caminho e áreas bloqueadas.
game/Viewport.java	Converte coordenadas entre tela e mapa.
game/GameEngine.java	Estado da partida e regras de colocação, combate e resultado.
ui/BattlefieldView.java	Canvas do mapa e eventos de toque.
ui/CampaignPager.java	Carrossel das três regiões.
ui/*Renderer + VisualClock	Desenho de heróis, portal, ambiente e tempo decorativo.
res/	XML das telas, PNG, vetores, textos, dimensões e tema.

Entrada do Android

app/src/main/AndroidManifest.xml:16-25

```
16              android:name=".MainActivity"
17              android:exported="true"
18              android:screenOrientation="sensorLandscape"
19              android:windowSoftInputMode="adjustResize">
20              <intent-filter>
21                  <action android:name="android.intent.action.MAIN"/>
22                  <category android:name="android.intent.category.LAUNCHER"/>
23              </intent-filter>
24          </activity>
25      </application>
```

O filtro MAIN/LAUNCHER faz o ícone abrir MainActivity. O projeto usa uma Activity para o menu e o combate, trocando o conteúdo de um FrameLayout.

> Exercício 1 Abra MainActivity.java e localize onCreate(), showMenu(), showPreparation() e startBattle(). Escreva em uma frase a função de cada método.

2. Como a tela muda

A navegação é uma sequência de chamadas da MainActivity. Primeiro ela cria um contêiner vazio; depois escolhe qual XML deve ocupar esse espaço.

Criação do contêiner

app/src/main/java/com/example/marveldefence/MainActivity.java:101-106

```
101     @Override
102     protected void onCreate(Bundle savedInstanceState) {
103         super.onCreate(savedInstanceState);
104         EdgeToEdge.enable(this);
105         setContentView(R.layout.activity_main);
106         root = findViewById(R.id.main);
```

R.layout.activity_main aponta para res/layout/activity_main.xml, que contém um FrameLayout com id main. A Activity guarda esse contêiner em root.

Troca do XML visível

app/src/main/java/com/example/marveldefence/MainActivity.java:161-176

```
161     private void showScreen(int layout) {
162         stopFrames();
163         preparation = null;
164         teamButtons.clear();
165         teamCards.clear();
166         root.removeAllViews();
167         getLayoutInflater().inflate(layout, root, true);
168         WindowInsetsControllerCompat bars = WindowCompat.getInsetsController(getWindow(), root);
169
    | bars.setSystemBarsBehavior(WindowInsetsControllerCompat.BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE);
170         if (screen == Screen.MENU) {
171             root.setPadding(0, 0, 0, 0);
172             bars.hide(WindowInsetsCompat.Type.systemBars());
173         } else
174             bars.show(WindowInsetsCompat.Type.systemBars());
175         ViewCompat.requestApplyInsets(root);
176     }
```

showScreen() para o ciclo anterior, remove as Views e infla outro layout dentro de root. No menu, carrega screen_campaign_menu.xml. Na partida, carrega screen_battle.xml. O pop-up de preparação é adicionado sobre o menu, sem criar outra Activity.

Preparação sobre a campanha

app/src/main/java/com/example/marveldefence/MainActivity.java:209-220

```
209     private void showPreparation() {
210         if (screen != Screen.MENU || preparation != null)
211             return;
212         CampaignPager pager = findViewById(R.id.campaign_pager);
213         if (!pager.canPrepareMission())
214             return;
215         pager.suspendEnvironment(true);
216         preparation = getLayoutInflater().inflate(R.layout.popup_mission, root, false);
217         root.addView(preparation);
218         root.getChildAt(0).setAlpha(.5f);
219         root.getChildAt(0).setRenderEffect(RenderEffect.createBlurEffect(dp(4), dp(4),
    | Shader.TileMode.CLAMP));
220
    | root.getChildAt(0).setImportantForAccessibility(View.IMPORTANT_FOR_ACCESSIBILITY_NO_HIDE_DESCEND
    | ANTS);
```

A primeira filha de root continua sendo o menu. O pop-up é acrescentado como outra filha; o menu recebe escurecimento e desfoque.

> Exercício 2 Siga showMenu() -> showPreparation() -> startBattle(). Anote em qual método o GameEngine passa a existir.

3. XML, recursos R e Views reais

O XML descreve o estado inicial de cada tela. O Android compila recursos em identificadores R. No Java, findViewById encontra um controle pelo id; o listener define o que acontece ao tocar.

Campo personalizado dentro do layout

app/src/main/res/layout/screen_battle.xml:1-5

```
1  <FrameLayout
2      xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent">
```

O botão ganha comportamento no Java

app/src/main/java/com/example/marveldefence/MainActivity.java:423-432

```
423          findViewById(R.id.battle_back).setOnClickListener(v -> leaveBattle());
424          findViewById(R.id.confirm_placement).setOnClickListener(v -> {
425              if (pendingHero != null && hasPosition && engine.place(pendingHero, pendingX,
     | pendingY))
426                  cancelPlacement();
427              else
428                  updatePlacementUi();
429              updateBattleUi();
430          });
431          findViewById(R.id.cancel_placement).setOnClickListener(v -> cancelPlacement());
432          findViewById(R.id.start_wave).setOnClickListener(v -> { engine.startWave();
     | updateBattleUi(); });
```

Em screen_battle.xml, @+id cria um identificador; @drawable aponta para fundo/vetor; @string aponta para texto; @style reutiliza propriedades. Alguns cartões são criados em Java. Por isso, o preview de XML pode aparecer incompleto.

Diretório em res	O que contém
layout/	activity_main, capa, campanha, pop-up e combate.
drawable-nodpi/	PNG do mapa, campanhas, barcos, logo e frente do ícone.
drawable/	Vetores, fundos, botões e seletores de estado.
values/	strings.xml, colors.xml, dimens.xml, styles.xml, themes.xml.
values-night/	Variante do tema para modo noturno.
mipmap-anydpi/	Definição do ícone adaptativo do launcher.
xml/	Arquivos de configuração de backup/extração.

> Exercício 3 Encontre start_wave em screen_battle.xml, depois procure R.id.start_wave em MainActivity.java e R.string.start_wave em strings.xml.

4. Herói: definição, unidade e arte

Há três conceitos que não devem ser confundidos. HeroCatalog.Hero é a definição compartilhada; GameEngine.Unit é o herói já colocado, com posição e melhorias; HeroRenderer desenha a aparência provisória. A ficha Comic Vine é separada do combate.

Campos de uma definição de herói

app/src/main/java/com/example/marveldefence/data/HeroCatalog.java:12-27

```
12      public static final class Hero {
13          public final String id, name, role;
14          public final int comicVineId, color, cost;
15          public final double damage, range, interval;
16          public final Attack attack;
17          Hero(String id, int apiId, String name, String role, int color, int cost,
18              double damage, double range, double interval, Attack attack) {
19              this.id = id;
20              this.comicVineId = apiId;
21              this.name = name;
22              this.role = role;
23              this.color = color;
24              this.cost = cost;
25              this.damage = damage;
26              this.range = range;
27              this.interval = interval;
```

Exemplo: os seis heróis da demonstração

app/src/main/java/com/example/marveldefence/data/HeroCatalog.java:31-39

```
31      public static final List<Hero> ALL = Collections.unmodifiableList(Arrays.asList(
32          new Hero("iron_man", 1455, "Homem de Ferro", "Repulsor • alvo único", 0xffe65140, 320, 18,
    |  170, .7, Attack.SINGLE),
33          new Hero("spider_man", 1443, "Homem-Aranha", "Teia • desaceleração", 0xffef5265, 250, 8,
    | 145, .65, Attack.SLOW),
34          new Hero("hulk", 2267, "Hulk", "Impacto • área próxima", 0xff78bd46, 360, 32, 100, 1.2,
    | Attack.AREA),
35          new Hero("thor", 2268, "Thor", "Trovão • área", 0xff82c9ef, 380, 22, 160, 1.0,
    | Attack.AREA),
36          new Hero("captain", 1442, "Capitão América", "Escudo • alvo único", 0xff498dde, 270, 16,
    | 145, .8, Attack.SINGLE),
37          new Hero("widow", 3200, "Viúva Negra", "Precisão • ataque rápido", 0xffbd80ec, 220, 10,
    | 150, .4, Attack.SINGLE)));
38      public static Hero find(String id) {
39          for (Hero hero : ALL)
```

No primeiro registro, iron_man custa 320, tem dano base 18, alcance 170, intervalo 0,7 s e ataque SINGLE. comicVineId identifica sua ficha; os números do jogo são locais e provisórios.

Estado próprio da unidade colocada

app/src/main/java/com/example/marveldefence/game/GameEngine.java:26-50

```
26      public static final class Unit {
27          public final HeroCatalog.Hero hero;
28          public final double x, y;
29          private int damageLevel, rangeLevel, invested;
30          private double cooldown;
31          Unit(HeroCatalog.Hero hero, double x, double y) {
32              this.hero = hero;
33              this.x = x;
34              this.y = y;
35              invested = hero.cost;
36          }
37          public double damage() {
38              return hero.damage * (1 + .4 * damageLevel);
39          }
40          public double range() {
41              return hero.range + 25 * rangeLevel;
42          }
43          public int damageLevel() {
44              return damageLevel;
45          }
46          public int rangeLevel() {
47              return rangeLevel;
48          }
49          public int sellValue() {
50              return (int) Math.floor(invested * GameConfig.SELL_FRACTION);
```

damageLevel e rangeLevel são da Unit. Melhorar uma unidade não altera o Hero original do catálogo. invested soma colocação e melhorias para calcular o valor de venda.

Selecionar até três ids para a equipe

app/src/main/java/com/example/marveldefence/MainActivity.java:286-302

```
286                 teamButtons.put(hero.id, select);
287             }
288         }
289         findViewById(R.id.mission_powers).setOnClickListener(v -> {
290             String[] names = new String[HeroCatalog.ALL.size()];
291             for (int i = 0; i < names.length; i++) names[i] = HeroCatalog.ALL.get(i).name;
292             new AlertDialog.Builder(this).setTitle(R.string.mission_powers).setItems(names,
    | (dialog, index) -> showProfile(HeroCatalog.ALL.get(index))).setNegativeButton(R.string.cancel,
    | null).show();
293         });
294         findViewById(R.id.deploy_button).setOnClickListener(v -> {
295             if (team.isEmpty())
296                 toast(R.string.select_team_first);
297             else
298                 startBattle();
299         });
300         Button sync = findViewById(R.id.sync_button);
301         sync.setEnabled(!repository.loading());
302         if (repository.loading())
```

A equipe guarda ids; selecionar um cartão no pop-up ainda não cria uma Unit. A Unit nasce apenas em GameEngine.place(). O emblema do controle vem de MainActivity.heroEmblem(); o desenho no mapa vem de HeroRenderer.draw().

> Exercício 4 Escolha um herói no catálogo. Anote id, custo, dano, alcance, intervalo e tipo de ataque. Depois encontre seu emblema em heroEmblem().

5. Imagem do cenário e geometria jogável

manhattan_map.png é a imagem visível. MapData representa, separadamente, rua, árvores e polígonos bloqueados. Suas coordenadas foram traçadas sobre a arte de 1672 x 941 pixels e convertidas para um mundo lógico de largura 1000.

Dimensões e escala do mundo

app/src/main/java/com/example/marveldefence/game/MapData.java:9-15

```
 9  public final class MapData {
10      public static final double IMAGE_WIDTH = 1672, IMAGE_HEIGHT = 941;
11      private static final double PIXEL_SCALE = 1000 / IMAGE_WIDTH;
12      public static final class Point {
13          public final double x, y;
14          public Point(double x, double y) {
15              this.x = x;
```

Começo da geometria da missão

app/src/main/java/com/example/marveldefence/game/MapData.java:55-67

```
55      public MapData() {
56          this(1000, IMAGE_HEIGHT * PIXEL_SCALE, 68, towerRoute(),
57              Arrays.asList(
58                  tree(112, 48, 42), tree(320, 128, 39), tree(580, 162, 48),
59                  tree(725, 60, 37), tree(1070, 155, 48), tree(1300, 258, 42),
60                  tree(1612, 418, 47), tree(605, 425, 34), tree(876, 414, 43),
61                  tree(939, 466, 36), tree(975, 513, 28), tree(125, 520, 47),
62                  tree(450, 728, 49), tree(1340, 630, 53), tree(1608, 550, 41),
63                  tree(1530, 740, 39), tree(1640, 745, 51), tree(1410, 854, 49),
64                  tree(1030, 910, 47), tree(1520, 635, 48)),
65              Arrays.asList(
66                  area("water", 0, 548, 95, 638, 272, 736, 502, 853, 700, 941, 0, 941),
67                  area("water", 0, 0, 380, 0, 220, 110, 0, 180),
```

O construtor escolhe largura lógica 1000, altura proporcional, largura do caminho 68, a rota da torre e obstáculos. Alterar apenas o PNG não altera a geometria de colisão.

Bloqueio e distância até a rua

app/src/main/java/com/example/marveldefence/game/MapData.java:129-156

```
129     public boolean blocked(double x, double y, double radius) {
130         for (Obstacle o : obstacles)
131             if (Math.hypot(x - o.x, y - o.y) <= radius + o.radius)
132                 return true;
133         for (BlockedArea area : blockedAreas)
134             if (area.intersects(x, y, radius))
135                 return true;
136         return false;
137     }
138     public Point position(double progress) {
139         double remaining = Math.max(0, progress);
140         for (int i = 1; i < path.size(); i++) {
141             Point a = path.get(i - 1), b = path.get(i);
142             double segment = distance(a, b);
143             if (segment == 0)
144                 continue;
145             if (remaining <= segment) {
146                 double t = remaining / segment;
147                 return new Point(a.x + (b.x - a.x) * t, a.y + (b.y - a.y) * t);
148             }
149             remaining -= segment;
150         }
151         return path.get(path.size() - 1);
152     }
153     public double distanceToPath(double x, double y) {
154         double best = Double.POSITIVE_INFINITY;
155         for (int i = 1; i < path.size(); i++) best = Math.min(best, segmentDistance(x, y,
    | path.get(i - 1), path.get(i)));
156         return best;
```

blocked() testa árvores circulares e áreas poligonais. distanceToPath() mede a distância até os segmentos da rua. position(progress) percorre os mesmos segmentos para mover os drones.

Viewport: tela e mundo usam escalas diferentes

Conversão de ida e volta

app/src/main/java/com/example/marveldefence/game/Viewport.java:12-26

```
12      public void cover(double screenWidth, double screenHeight, double worldWidth, double
    | worldHeight) {
13          scale = Math.max(.0001, Math.max(screenWidth / worldWidth, screenHeight / worldHeight));
14          left = (screenWidth - worldWidth * scale) / 2;
15          top = (screenHeight - worldHeight * scale) / 2;
16      }
17      public double worldX(double screenX) {
18          return (screenX - left) / scale;
19      }
20      public double worldY(double screenY) {
21          return (screenY - top) / scale;
22      }
23      public double screenX(double worldX) {
24          return left + worldX * scale;
25      }
26      public double screenY(double worldY) {
```

cover() amplia o mapa sem distorcer para preencher a área de combate; bordas podem ser cortadas. worldX/worldY fazem a conversão inversa para que o toque continue alinhado ao desenho. A prévia usa fit(), que tenta mostrar o mapa inteiro.

> Exercício 5 Localize manhattan_map.png, MapData.towerRoute(), Viewport.cover() e BattlefieldView.onDraw(). Explique por que imagem e geometria precisam concordar.

6. Siga um toque até colocar um herói

O fluxo é: cartão do herói -> beginPlacement() -> toque em BattlefieldView -> mapTouched() -> placement() -> confirmar -> place(). A prévia visual não cobra créditos.

O toque em pixels vira coordenadas do mapa

app/src/main/java/com/example/marveldefence/ui/BattlefieldView.java:153-168

```
153     @Override
154     public boolean onTouchEvent(MotionEvent event) {
155         if (listener == null || engine == null || engine.finished())
156             return false;
157         double x = viewport.worldX(event.getX()), y = viewport.worldY(event.getY());
158         boolean inside = x >= 0 && y >= 0 && x <= map.width && y <= map.height;
159         if (event.getAction() == MotionEvent.ACTION_DOWN)
160             return inside;
161         if (event.getAction() == MotionEvent.ACTION_UP) {
162             performClick();
163             if (inside)
164                 listener.touched(x, y);
165             return true;
166         }
167         return true;
168     }
```

A Activity guarda uma posição pendente

app/src/main/java/com/example/marveldefence/MainActivity.java:454-468

```
454     private void beginPlacement(String id) {
455         closeUpgrade();
456         pendingHero = id;
457         hasPosition = false;
458         battlefield.pending(id, 0, 0, false);
459         updateBattleUi();
460     }
461     private void mapTouched(double x, double y) {
462         if (pendingHero != null) {
463             pendingX = x;
464             pendingY = y;
465             hasPosition = true;
466             battlefield.pending(pendingHero, x, y, true);
467             updatePlacementUi();
468             return;
```

pendingHero indica qual herói o jogador deseja posicionar. O toque no mapa só escolhe x/y e mostra a prévia. updatePlacementUi() consulta a validação do motor e habilita Confirmar somente quando o resultado é VALID.

Todas as razões para recusar uma posição

app/src/main/java/com/example/marveldefence/game/GameEngine.java:164-186

```
164     public Placement placement(String id, double x, double y) {
165         if (finished())
166             return Placement.FINISHED;
167         if (!team.contains(id))
168             return Placement.NOT_IN_TEAM;
169         if (placed(id))
170             return Placement.ALREADY_PLACED;
171         double r = GameConfig.HERO_RADIUS;
172         if (!Double.isFinite(x) || !Double.isFinite(y) || x < r || y < r || x > map.width - r || y
    |  > map.height - r)
173             return Placement.OUTSIDE;
174         double d = map.distanceToPath(x, y);
175         if (d < map.pathWidth / 2 + r + GameConfig.PATH_MARGIN)
176             return Placement.PATH;
177         if (d > map.pathWidth / 2 + GameConfig.PLACEMENT_BAND)
178             return Placement.TOO_FAR;
179         if (map.blocked(x, y, r))
180             return Placement.OBSTACLE;
181         for (Unit u : units)
182             if (Math.hypot(x - u.x, y - u.y) < 2 * r + GameConfig.PATH_MARGIN)
183                 return Placement.OVERLAP;
184         if (credits < HeroCatalog.find(id).cost)
185             return Placement.NO_CREDITS;
186         return Placement.VALID;
```

Cobrança e criação da Unit

app/src/main/java/com/example/marveldefence/game/GameEngine.java:188-195

```
188     public boolean place(String id, double x, double y) {
189         if (placement(id, x, y) != Placement.VALID)
190             return false;
191         HeroCatalog.Hero hero = HeroCatalog.find(id);
192         credits -= hero.cost;
193         units.add(new Unit(hero, x, y));
194         return true;
195     }
```

O custo é descontado somente depois de placement() retornar VALID. Cancelar antes da confirmação não paga nada. GameEngine é a fonte da decisão; a View apenas indica a decisão com cor.

> Exercício 6 Coloque breakpoints em beginPlacement(), mapTouched(), placement() e place(). Observe pendingHero, x/y e credits em cada parada.

7. O relógio da partida

Choreographer chama MainActivity.doFrame() a cada oportunidade de desenhar. A Activity mede o tempo passado, mas sempre entrega ao motor passos fixos de 1/60 s. O desenho pode ocorrer em outra frequência sem mudar o tamanho de cada passo da simulação.

Acúmulo de tempo, atualização e desenho

app/src/main/java/com/example/marveldefence/MainActivity.java:72-98

```
72      private final Choreographer.FrameCallback frame = new Choreographer.FrameCallback() {
73          @Override
74          public void doFrame(long time) {
75              if (!ticking || !resumed || screen != Screen.BATTLE || engine == null)
76                  return;
77              if (lastFrame != 0 && !engine.paused()) {
78                  accumulator += Math.min(GameConfig.MAX_FRAME_TIME, (time - lastFrame) /
    | 1_000_000_000.0) * speed;
79                  int steps = 0;
80                  while (accumulator >= GameConfig.FIXED_STEP && steps++ < 12) {
81                      engine.update(GameConfig.FIXED_STEP);
82                      accumulator -= GameConfig.FIXED_STEP;
83                  }
84              } else
85                  accumulator = 0;
86              lastFrame = time;
87              battlefield.invalidate();
88              if (time - lastHud > 150_000_000L) {
89                  updateBattleUi();
90                  lastHud = time;
91              }
92              if (engine.finished()) {
93                  stopFrames();
94                  showResult();
95                  return;
96              }
97              Choreographer.getInstance().postFrameCallback(this);
98          }
```

speed vale 1 ou 2. MAX_FRAME_TIME limita tempo de um quadro e o laço limita a 12 passos. invalidate() pede novo desenho; updateBattleUi() atualiza o HUD em intervalos. Quando o app vai para segundo plano, o ciclo para e não compensa o tempo ausente.

Parâmetros do ciclo, colocação e ondas

app/src/main/java/com/example/marveldefence/game/GameConfig.java:5-11

```
5       public static final int TEAM_LIMIT = 3, INITIAL_LIVES = 100, INITIAL_CREDITS = 1000;
6       public static final int WAVE_BONUS = 100, UPGRADE_LIMIT = 3;
7       public static final double HERO_RADIUS = 21, PATH_MARGIN = 9, PLACEMENT_BAND = 180;
8       public static final double FIXED_STEP = 1.0 / 60, MAX_FRAME_TIME = .1;
9       public static final double SELL_FRACTION = .7, SLOW_FACTOR = .55, SLOW_DURATION = 1.5;
10      public static final int[] WAVE_COUNTS = {8, 12, 16, 20, 26};
11      public static final double[] SPAWN_INTERVALS = {.85, .7, .6, .5, .42};
```

> Exercício 7 Explique por que engine.update() recebe sempre GameConfig.FIXED_STEP, e o que mudaria se speed passasse de 1 para 2.

8. O que acontece dentro de uma onda

Leia GameEngine.update() em quatro blocos: 1) gerar inimigos; 2) unidades atacarem; 3) remover mortos ou mover sobreviventes; 4) verificar derrota ou fim da onda.

Início da onda e surgimento

app/src/main/java/com/example/marveldefence/game/GameEngine.java:217-242

```
217     public boolean startWave() {
218         if (status != Status.READY || wave >= GameConfig.WAVE_COUNTS.length)
219             return false;
220         wave++;
221         spawned = 0;
222         spawnTimer = 0;
223         paused = false;
224         status = Status.RUNNING;
225         return true;
226     }
227     public void update(double dt) {
228         if (paused || status != Status.RUNNING || !Double.isFinite(dt) || dt <= 0)
229             return;
230         simulationTime += dt;
231         for (int i = effects.size() - 1; i >= 0; i--)
232             if ((effects.get(i).remaining -= dt) <= 0)
233                 effects.remove(i);
234         spawnTimer -= dt;
235         while (spawned < GameConfig.WAVE_COUNTS[wave - 1] && spawnTimer <= 0) {
236             int type = wave == 1 ? 0 : wave >= 3 && spawned % 4 == 3 ? 2
237                 : spawned % 3 == 1                                   ? 1
238                                                                      : 0;
239             enemies.add(new Enemy(type, wave - 1, map));
240             spawned++;
241             spawnTimer += GameConfig.SPAWN_INTERVALS[wave - 1];
242         }
```

startWave() só funciona em READY. WAVE_COUNTS diz quantos drones devem nascer; SPAWN_INTERVALS define o ritmo. spawned conta os drones já criados.

Cooldown, escolha de alvo e ataque

app/src/main/java/com/example/marveldefence/game/GameEngine.java:243-267

```
243         for (Unit unit : units) {
244             unit.cooldown = Math.max(0, unit.cooldown - dt);
245             if (unit.cooldown > 0)
246                 continue;
247             Enemy target = null;
248             for (Enemy e : enemies)
249                 if (e.health > 0 && Math.hypot(e.x - unit.x, e.y - unit.y) <= unit.range()
250                     && (target == null || e.progress > target.progress))
251                     target = e;
252             if (target == null)
253                 continue;
254             effects.add(new Effect(unit, target));
255             if (unit.hero.attack == HeroCatalog.Attack.AREA) {
256                 for (Enemy e : enemies)
257                     if (e.health > 0 && Math.hypot(e.x - target.x, e.y - target.y) <= 48
258                         && Math.hypot(e.x - unit.x, e.y - unit.y) <= unit.range())
259                         e.health -= unit.damage();
260             } else {
261                 target.health -= unit.damage();
262                 // Aplicar lentidão novamente renova a duração, sem acumular o multiplicador.
263                 if (unit.hero.attack == HeroCatalog.Attack.SLOW)
264                     target.slowRemaining = GameConfig.SLOW_DURATION;
265             }
266             unit.cooldown = unit.hero.interval;
267         }
```

Uma unidade escolhe, dentro do seu alcance, o inimigo vivo com maior progress no caminho. SINGLE atinge um alvo; AREA atinge perto do alvo; SLOW renova uma lentidão temporária. O dano é imediato; Effect registra uma linha visual curta.

Recompensa, escape e conclusão

app/src/main/java/com/example/marveldefence/game/GameEngine.java:268-297

```
268         for (int i = enemies.size() - 1; i >= 0; i--) {
269             Enemy e = enemies.get(i);
270             if (e.health <= 0) {
271                 credits += e.reward;
272                 kills++;
273                 enemies.remove(i);
274                 continue;
275             }
276             double slowedTime = Math.min(dt, e.slowRemaining);
277             e.progress += e.speed * (slowedTime * GameConfig.SLOW_FACTOR + dt - slowedTime);
278             e.slowRemaining = Math.max(0, e.slowRemaining - dt);
279             MapData.Point point = map.position(e.progress);
280             e.x = point.x;
281             e.y = point.y;
282             if (e.progress >= map.length) {
283                 lives -= e.escapeDamage;
284                 enemies.remove(i);
285             }
286         }
287         if (lives <= 0) {
288             lives = 0;
289             status = Status.LOST;
290             effects.clear();
291             return;
292         }
293         if (spawned == GameConfig.WAVE_COUNTS[wave - 1] && enemies.isEmpty()) {
294             credits += GameConfig.WAVE_BONUS;
295             effects.clear();
296             status = wave == GameConfig.WAVE_COUNTS.length ? Status.WON : Status.READY;
297         }
```

Um inimigo morto dá créditos uma vez. Um inimigo que termina o trajeto reduz vidas e não dá recompensa. Derrota é verificada antes da conclusão da onda. Após todos nascerem e nenhum permanecer ativo, há bônus; a próxima onda exige novo toque. A quinta concluída vira WON.

> Exercício 8 Marque os quatro blocos dentro de update(). Identifique onde credits, lives, kills e status são alterados.

9. Como o estado vira pixels

Depois de cada atualização do motor, o campo é redesenhado e os controles Android recebem os números novos. Vale ler essas duas saídas separadamente.

Ordem de desenho no Canvas

app/src/main/java/com/example/marveldefence/ui/BattlefieldView.java:85-120

```
85      @Override
86      protected void onDraw(Canvas canvas) {
87          super.onDraw(canvas);
88          canvas.drawColor(0xff091928);
89          canvas.save();
90          canvas.translate((float) viewport.left, (float) viewport.top);
91          canvas.scale((float) viewport.scale, (float) viewport.scale);
92          canvas.clipRect(terrainBounds);
93          canvas.drawBitmap(terrain, null, terrainBounds, terrainPaint);
94          canvas.save();
95          canvas.scale((float) (map.width / EnvironmentScene.WIDTH), (float) (map.height /
    | EnvironmentScene.HEIGHT));
96          environment.draw(canvas, visualSeconds);
97          canvas.restore();
98          portal.draw(canvas, portalDegrees);
99          if (engine != null) {
100             if (selected != null)
101                 range(canvas, selected.x, selected.y, selected.range(), 0x337bdccd, 0xffa6f5e0);
102             if (pendingHero != null && hasPosition) {
103                 boolean valid = engine.placement(pendingHero, pendingX, pendingY) ==
    | GameEngine.Placement.VALID;
104                 range(canvas, pendingX, pendingY, HeroCatalog.find(pendingHero).range, valid ?
    | 0x4460d98c : 0x44ef5265, valid ? 0xffb3f18c : 0xffff6f82);
105             }
106             for (GameEngine.Enemy enemy : engine.enemies()) drawDrone(canvas, enemy);
107             for (GameEngine.Unit unit : engine.units()) heroes.draw(canvas, unit.hero, (float)
    | unit.x, (float) unit.y + 2, 13);
108             for (GameEngine.Effect effect : engine.effects()) {
109                 paint.setColor(effect.color);
110                 paint.setStrokeWidth(4);
111                 paint.setStyle(Paint.Style.STROKE);
112                 canvas.drawLine((float) effect.x1, (float) effect.y1, (float) effect.x2, (float)
    | effect.y2, paint);
113                 paint.setStyle(Paint.Style.FILL);
114                 canvas.drawCircle((float) effect.x2, (float) effect.y2, 7, paint);
115             }
116             if (pendingHero != null && hasPosition)
117                 heroes.draw(canvas, HeroCatalog.find(pendingHero), (float) pendingX, (float)
    | pendingY + 2, 13);
118         }
119         paint.setStyle(Paint.Style.FILL);
120         canvas.restore();
```

A View aplica translate e scale do Viewport; depois desenha mapa, ambiente, portal, inimigos, unidades, efeitos e prévia. Todas essas partes usam a mesma transformação. A View lê o motor; não decide dano ou vitória.

HUD e botões Android leem o motor

app/src/main/java/com/example/marveldefence/MainActivity.java:630-645

```
630     private void updateBattleUi() {
631         if (screen != Screen.BATTLE || engine == null)
632             return;
633         ((TextView) findViewById(R.id.lives)).setText(getString(R.string.hud_lives,
    | engine.lives()));
634         ((TextView) findViewById(R.id.credits)).setText(getString(R.string.hud_credits,
    | engine.credits()));
635         ((TextView) findViewById(R.id.wave)).setText(getString(R.string.hud_wave, engine.wave(),
    | GameConfig.WAVE_COUNTS.length));
636         boolean ready = engine.status() == GameEngine.Status.READY;
637         Button start = findViewById(R.id.start_wave);
638         start.setEnabled(ready);
639         start.setText(R.string.combat_play);
640         start.setContentDescription(getString(ready ? R.string.start_wave :
    | R.string.running_wave));
641         Button pause = findViewById(R.id.pause_button);
642         pause.setEnabled(engine.status() == GameEngine.Status.RUNNING);
643         pause.setText(engine.paused() ? R.string.combat_play : R.string.combat_pause);
644         pause.setContentDescription(getString(engine.paused() ? R.string.resume :
    | R.string.pause));
645         ((Button) findViewById(R.id.speed_button)).setText(speed == 1 ? R.string.speed_normal :
    | R.string.speed_fast);
```

updateBattleUi() lê lives, credits, wave e status. A tela mistura Canvas para o campo com TextView/Button para os controles. Ao mudar o estado do motor, o próximo desenho e a atualização do HUD mostram o resultado.

> Exercício 9 Quando um drone morre, encontre onde o motor aumenta credits e onde a Activity mostra o novo valor.

10. Fichas, ambiente e ciclo de vida

Comic Vine não controla o balanceamento

Campos informativos da ficha

app/src/main/java/com/example/marveldefence/data/ComicVineRepository.java:24-43

```
24  public final class ComicVineRepository {
25      public static final class Profile {
26          public final String name, description, imageUrl, sourceUrl;
27          public final List<String> powers;
28          Profile(JSONObject json) {
29              name = clean(json, "name");
30              description = clean(json, "deck");
31              sourceUrl = clean(json, "site_detail_url");
32              JSONObject image = json.optJSONObject("image");
33              imageUrl = image == null ? "" : clean(image, "small_url");
34              ArrayList<String> values = new ArrayList<>();
35              JSONArray array = json.optJSONArray("powers");
36              if (array != null)
37                  for (int i = 0; i < array.length(); i++) {
38                      JSONObject power = array.optJSONObject(i);
39                      if (power != null && !clean(power, "name").isEmpty())
40                          values.add(clean(power, "name"));
41                  }
42              powers = Collections.unmodifiableList(values);
43          }
```

Atualização manual, intervalo e cache

app/src/main/java/com/example/marveldefence/data/ComicVineRepository.java:86-109

```
86      public boolean refresh(Callback callback) {
87          long now = System.currentTimeMillis();
88          long lastAttempt = context.getSharedPreferences("comicvine",
    | Context.MODE_PRIVATE).getLong("last_attempt", 0);
89          // São seis registros por atualização; o intervalo de dois minutos limita a menos de 200
    | consultas por hora.
90          if (closed || loading || !hasKey() || now - lastAttempt < 120000 && lastAttempt != 0)
91              return false;
92          loading = true;
93          context.getSharedPreferences("comicvine",
    | Context.MODE_PRIVATE).edit().putLong("last_attempt", now).apply();
94          worker.execute(() -> {
95              Map<String, Profile> updated = new HashMap<>();
96              int failures = 0;
97              for (HeroCatalog.Hero hero : HeroCatalog.ALL) {
98                  if (Thread.currentThread().isInterrupted())
99                      return;
100                 try {
101                     JSONObject result = fetch(hero.comicVineId);
102                     // Cada identidade jogável é fixa e tem a editora conferida; não é escolhida
    | por busca.
103                     if (result.optInt("id") != hero.comicVineId ||
    | result.optString("name").isEmpty())
104                         throw new Exception("Identity");
105                     JSONObject publisher = result.optJSONObject("publisher");
106                     if (publisher == null || !publisher.optString("name").startsWith("Marvel"))
107                         throw new Exception("Publisher");
108                     updated.put(hero.id, new Profile(result));
109                     context.getSharedPreferences("comicvine",
    | Context.MODE_PRIVATE).edit().putString(hero.id, result.toString()).apply();
```

O repositório inicia com fichas incluídas em assets/comicvine_profiles.json e lê cache local. Atualização é manual, em uma fila de trabalho, fora do combate. O resultado volta à thread principal para atualizar a interface. Não copie a chave local de debug para código ou documentação.

Animações decorativas têm outro tempo

Relógio visual independente

app/src/main/java/com/example/marveldefence/ui/VisualClock.java:15-39

```
15          this.listener = listener;
16      }
17      void running(boolean enabled) {
18          if (running == enabled)
19              return;
20          running = enabled;
21          previous = 0;
22          if (enabled)
23              Choreographer.getInstance().postFrameCallback(this);
24          else
25              Choreographer.getInstance().removeFrameCallback(this);
26      }
27      @Override
28      public void doFrame(long time) {
29          if (!running)
30              return;
31          if (previous != 0)
32              seconds += Math.min(.1, (time - previous) / 1_000_000_000.0);
33          previous = time;
34          listener.frame(seconds);
35          if (running)
36              Choreographer.getInstance().postFrameCallbackDelayed(this, 33);
37      }
38  }
```

Portal e ambiente não dependem de pausa nem de 2x. EnvironmentScene descreve regiões e parâmetros; EnvironmentRenderer desenha água, nuvens e barcos; CampaignImageView compõe efeitos sobre páginas da campanha.

O que ocorre quando o app sai de foco

Parada do ciclo e encerramento do repositório

app/src/main/java/com/example/marveldefence/MainActivity.java:749-769

```
749     @Override
750     protected void onResume() {
751         super.onResume();
752         resumed = true;
753         visualForeground(true);
754         startFrames();
755     }
756     @Override
757     protected void onPause() {
758         resumed = false;
759         stopFrames();
760         visualForeground(false);
761         if (engine != null)
762             engine.setPaused(true);
763         super.onPause();
764     }
765     @Override
766     protected void onDestroy() {
767         stopFrames();
768         repository.close();
769         super.onDestroy();
```

onPause() interrompe os frames, suspende efeitos visuais e pausa o motor. onDestroy() libera o repositório. A partida não é recuperada após recriar Activity ou processo; uma nova sessão começa no menu.

> Exercício 10 Compare os dois relógios: MainActivity.frame atualiza regras; VisualClock atualiza decoração. Liste o que cada um altera.

11. Testes e roteiro prático final

GameEngineTest.java testa lógica sem abrir Android: posicionamento, cobranças, intervalo de ataque, prioridade, lentidão, pausa, cinco ondas e reinício. BattleLayoutInstrumentedTest.java verifica parte da interface em Android. Build e testes não substituem inspeção visual real.

Pergunta	Onde procurar
Onde nasce uma partida nova?	MainActivity.startBattle()
Quem permite ou recusa posicionamento?	GameEngine.placement()
Quem cobra o custo?	GameEngine.place()
Quem traduz o toque?	Viewport + BattlefieldView.onTouchEvent()
Quem escolhe o alvo?	GameEngine.update()
Quem desenha o herói?	HeroRenderer.draw() via BattlefieldView
Quem atualiza os números da tela?	MainActivity.updateBattleUi()

Pergunta	Onde procurar
Onde estão as fichas informativas?	ComicVineRepository + assets/comicvine_profiles.json

Verificação registrada em docs/VALIDACAO.md: em 18/10/2026, o build debug e 16 testes do motor mais 1 teste básico passaram. Isso confirma compilação e lógica testada. A aparência em aparelho da revisão final é uma verificação separada.

Sequência completa para você narrar

Passo	Chamada ou mudança
1	Android -> MainActivity.onCreate() -> showMenu()
2	CampaignPager -> plataforma -> showPreparation()
3	Equipe -> startBattle() -> new GameEngine(new MapData(), team)
4	Escolher herói -> tocar mapa -> placement() -> place()
5	Iniciar onda -> Choreographer -> engine.update(1/60)
6	Spawn -> ataque -> movimento -> morte/escape -> READY/WON/LOST
7	BattlefieldView.onDraw() e MainActivity.updateBattleUi() mostram o estado

> Prática recomendada No Android Studio, abra esses métodos na ordem acima e use breakpoints em mapTouched(), placement(), place() e update(). Após colocar um herói e iniciar uma onda, acompanhe os valores no depurador.

Documentação complementar: docs/PROJETO.md registra decisões; docs/PROGRESSO.md registra etapas; docs/VALIDACAO.md separa o que foi implementado do que foi verificado. Para aprender o comportamento atual, priorize sempre o código executável.